

LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK (PBO)

***MEMBUAT FUNGSI CRUD (Create, Read, Update, Delete)
DENGAN DATABASE MYSQL
(service dan customer)***



OLEH :

NIA RAMADHANI
(2311531006)

DOSEN PENGUMPU : Nurfiah, S.ST, M.Kom.

**FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS**

1.1 Pendahuluan

Pada praktikum ini, saya diharapkan mampu mengimplementasikan fungsi CRUD (Create, Read, Update, Delete) untuk data pengguna menggunakan database MySQL. Praktikum ini bertujuan untuk melatih kemampuan saya dalam membuat tabel pengguna di MySQL, menghubungkan aplikasi Java dengan database, membuat tampilan GUI (Graphical User Interface) untuk CRUD pengguna, serta mengaplikasikan konsep Pemrograman Berorientasi Objek (PBO). Selain itu, saya juga akan mempelajari pembuatan dan penggunaan interface serta fungsi DAO (Data Access Object) yang dapat memisahkan logika akses data dari logika bisnis, meningkatkan modularitas, dan memungkinkan perubahan dalam pengelolaan data tanpa mempengaruhi bagian lain dari aplikasi. Alat-alat yang dibutuhkan untuk praktikum ini antara lain komputer yang telah terinstal JDK, Eclipse, MySQL atau XAMPP, serta MySQL Connector untuk menghubungkan Java dengan MySQL.

1.2 Tujuan

Tujuan praktikum ini yaitu mahasiswa mampu membuat fungsi CRUD data user menggunakan database MySQL, Adapun poin-poin praktikum yaitu :

- Mahasiswa mampu membuat table user pada database MySQL
- Mahasiswa mampu membuat koneksi Java dengan database MySQL
- Mahasiswa mampu membuat tampilan GUI CRUD user
- Mahasiswa mampu membuat dan mengimplementasikan interface
- Mahasiswa mampu membuat fungsi DAO (Data Access Object) dan mengimplementasikannya.
- Mahasiswa mampu membuat fungsi CRUD dengan menggunakan konsep Pemrograman Berorientasi Objek

1.3 Alat

- Computer / laptop yang telah terinstall JDK dan Eclipse
- MySQL / XAMPP
- MySQL connector atau Connector/J

1.4 Teori

XAMPP yaitu paket software yang terdiri dari Apache HTTP Server, MySQL, PHP dan Perl yang bersifat open source, xampp biasanya digunakan sebagai development environment dalam pengembangan aplikasi berbasis web secara localhost.

Apache berfungsi sebagai web server yang digunakan untuk menjalankan halaman web, MySQL digunakan untuk manajemen basis data dalam melakukan manipulasi data, PHP digunakan sebagai Bahasa pemrograman untuk membuat aplikasi berbasis web.

MySQL adalah sebuah relational database management system (RDBMS) open-source yang digunakan dalam pengelolaan database suatu aplikasi, MySQL ini dapat digunakan untuk menyimpan, mengelola dan mengambil data dalam format table.



Logo MySQL

MySQL Connection/j adalah driver yang digunakan untuk menghubungkan aplikasi berbasis java dengan database MySQL sehingga dapat berinteraksi seperti menyimpan, mengubah, mengambil dan menghapus data. Beberapa fungsi MySQL connector yaitu :

- Membuka koneksi ke database MySQL
- Mengirimkan permintaan SQL ke server MySQL
- Menerima hasil dari permintaan SQL
- Menutup koneksi ke database MySQL

DAO (Data Access Object) merupakan object yang menyediakan abstract interface terhadap beberapa method yang berhubungan dengan database seperti mengambil data (read), menyimpan data(create), menghapus data (delete), mengubah data(update). Tujuan penggunaan DAO yaitu :

- Meningkatkan modularitas yaitu memisahkan logika akses data dengan logika bisnis sehingga memudahkan untuk dikelola
- Meningkatkan reusabilitas yaitu DAO dapat digunakan Kembali
- Perubahan pada logika akses data dapat dilakukan tanpa mempengaruhi logika bisnis.

Interface dalam Bahasa java yaitu mendefinisikan beberapa method abstrak yang harus diimplementasikan oleh class yang akan menggunakannya.

CRUD (Create, Read, Update, Delete) merupakan fungsi dasar atau umum yang ada pada sebuah aplikasi yang mana fungsi ini dapat membuat, membaca, mengubah dan menghapus suatu data pada database aplikasi.

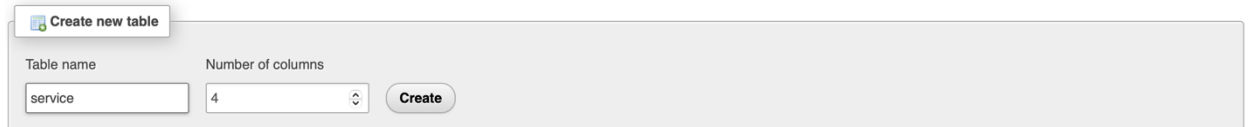
1.5 Langkah-langkah

XAMPP

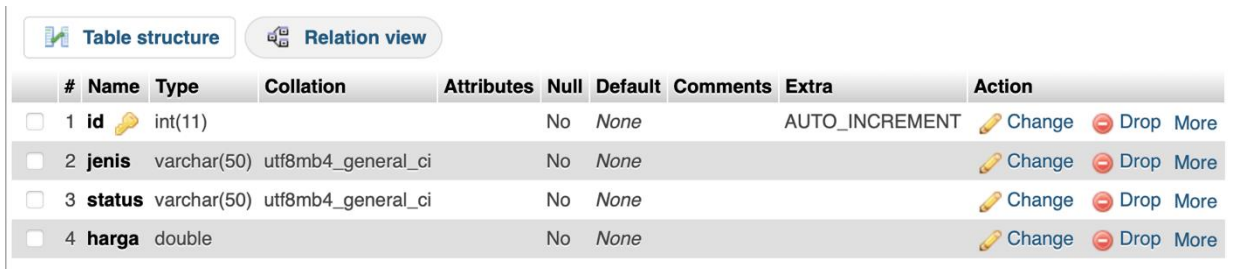
- Jalankan xampp dan aktifkan apache dan mysql

Membuat Database dan Table Service dan Table Costumer

- Buka <http://localhost/phpmyadmin>
- Buat table service dengan cara klik database laundry_apps dan buat table dengan nama service

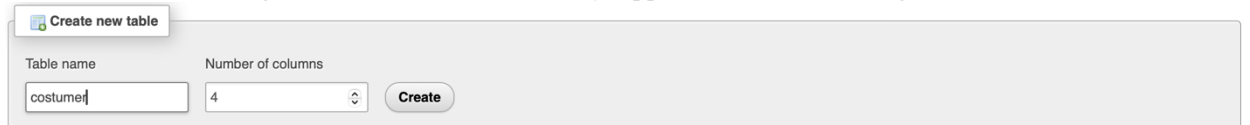


Klik create, kemudian isi tabel dengan 'id', 'jenis', 'status', dan 'harga'. Maka akan muncul seperti gambar dibawah ini.

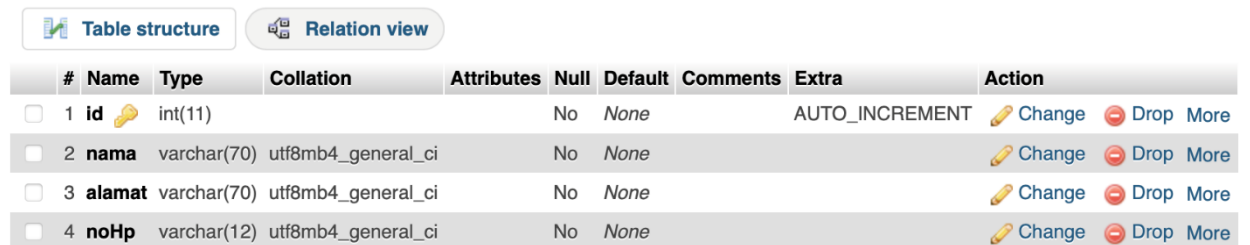


#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 jenis	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 status	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 harga	double			No	None			Change Drop More

- Buat table service dengan cara klik database laundry_apps dan buat table dengan nama Costumer



Klik create, kemudian isi tabel dengan 'id', 'jenis', 'status', dan 'harga'. Maka akan muncul seperti gambar dibawah ini.



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 nama	varchar(70)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 alamat	varchar(70)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 noHp	varchar(12)	utf8mb4_general_ci		No	None			Change Drop More

Membuat Tampilan CRUD Service dan Customer

- Buat file baru menggunakan JFrame pada package ui dengan nama ServiceFrame seperti gambar berikut ini.

Nama

Alamat

NoHP

Save **Update** **Delete** **Cancel**

ID	Name	Alamat	NoHP
1	NANA	jln gaja mada	0927383

Keterangan :

Component	Variable	Keterangan
JComboBox	comboJenis	Jenis
TextField	txtQuantity	Nilai Satuan
JComboBox	comboSatuan	Satuan
JComboBox	comboStatus	Status

TextField	txtHarga	Harga
Button	btnSave	Save
Button	btnUpdate	Update
Button	btnDelete	Delete
Button	btnCancel	Cancel
Table	tableService	Table Service

- Buat file baru menggunakan JFrame pada package ui dengan nama CostumerFrame seperti gambar berikut ini.

The image shows a Java Swing window titled "CostumerFrame". The window has a light gray background and a dark gray border. It contains three text input fields labeled "Nama", "Alamat", and "No HP". Below these fields are four buttons: "Save", "Update", "Delete", and "Cancel". At the bottom of the window is a large, empty rectangular area, likely intended for a table or list.

Keterangan :

Component	Variable	Keterangan
TextField	txtNama	Nama
TextField	txtAlamat	Alamat

TextField	txtNoHp	Nomor HP
Button	btnSave	Save
Button	btnUpdate	Update
Button	btnDelete	Delete
Button	btnCancel	Cancel
JTable	tableService	Table Service

Membuat Table Model

Table model user ini berguna untuk mengambil data dari database dan ditampilkan kedalam table.

- Buat file baru didalam package table dengan nama **TableService**

>  TableService.java

- Buat file baru didalam package table dengan nama **TableCustomer**

>  TableCustomer.java

kemudian isikan TableService dengan kode program berikut.

```
1 package table;
2
3 import java.util.List;
4
5
6
7 public class TableService extends AbstractTableModel {
8     private List<Service> serviceList;
9     private String[] columnNames = { "ID", "Jenis", "Satuan", "Status", "Harga" };
10
11
12     public TableService(List<Service> serviceList) {
13         this.serviceList = serviceList;
14     }
15
16     @Override
17     public int getRowCount() {
18         return serviceList.size();
19     }
20
21     @Override
22     public int getColumnCount() {
23         return columnNames.length;
24     }
25
26     @Override
27     public Object getValueAt(int rowIndex, int columnIndex) {
28         Service service = serviceList.get(rowIndex);
29         switch (columnIndex) {
30             case 0: return service.getId();
31             case 1: return service.getJenis();
32             case 2: return service.getSatuan();
33             case 3: return service.getStatus();
34             case 4: return service.getHarga();
35         }
36     }
37 }
```

kemudian isikan TableCustomer dengan kode program berikut.

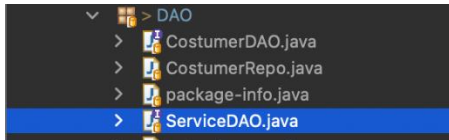
```
1 package table;
2
3 import java.util.List;
4
5
6
7 public class TableCustomer extends AbstractTableModel {
8     private List<Customer> customerList;
9     private String[] columnNames = { "ID", "Name", "Alamat", "NoHP" };
10
11
12 public TableCustomer(List<Customer> customerList) {
13     this.customerList = customerList;
14 }
15
16 @Override
17 public int getRowCount() {
18     return customerList.size();
19 }
20
21 @Override
22 public int getColumnCount() {
23     return columnNames.length;
24 }
25
26 @Override
27 public Object getValueAt(int rowIndex, int columnIndex) {
28     Customer customer = customerList.get(rowIndex);
29     switch (columnIndex) {
30         case 0: return customer.getId();
31         case 1: return customer.getNama();
32         case 2: return customer.getAlamat();
33         case 3: return customer.getNoHP();
34         default: return null;
```

Membuat Fungsi DAO

- Pada package DAO



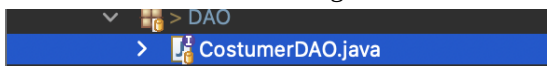
- Buat class Interface baru dengan nama **ServiceDAO**,



kemudian isikan dengan kode program berikut.

```
1 package DAO;
2
3 import java.util.List;
4
5
6
7 public interface ServiceDAO {
8     void save(Service service);
9     List<Service> show();
0     void update(Service service);
1     void delete(int id);
2     void delete(String id);
3
4 }
```

- Buat class Interface baru dengan nama **CostumerDAO**,



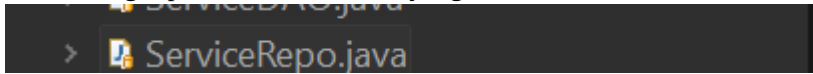
kemudian isikan dengan kode program berikut.

```
1 package DAO;
2
3 import java.util.List;
4
5
6
7
8 public interface CustomerDAO {
9     void save(Customer customer);
10    List<Customer> show();
11    void update(Customer customer);
12    void delete(int id);
13    void delete(String id);
14
15 }
```

Terdapat method **save**, **show**, **delete** dan **update**. Method pada class interface digunakan sebagai method utama yang wajib diimplementasikan pada class yang menggunakannya.

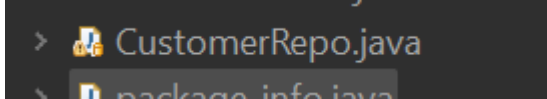
Menggunakan Fungsi DAO

- Buat class baru pada package DAO dengan nama **ServiceRepo** yang mana akan digunakan untuk mengimplementasikan DAO yang telah dibuat.



```
> ServiceRepo.java
```

- Buat class baru pada package DAO dengan nama **CostumerRepo** yang mana akan digunakan untuk mengimplementasikan DAO yang telah dibuat.



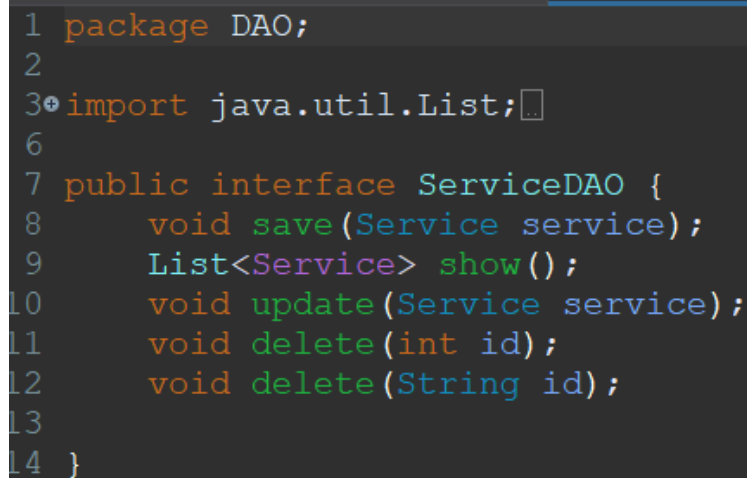
```
> CustomerRepo.java
```

- Implementasikan ServiceDao dengan kata kunci **implements**



```
12 public class ServiceRepo implements ServiceDAO {
```

- Masukkan kode di bawah ini



```
1 package DAO;
2
3 import java.util.List;
4
5
6
7 public interface ServiceDAO {
8     void save(Service service);
9     List<Service> show();
10    void update(Service service);
11    void delete(int id);
12    void delete(String id);
13
14 }
```

- Implementasikan CostumerDao dengan kata kunci **implements**



```
16 public class CostumerRepo implements CostumerDAO {
```

- Masukkan kode di bawah ini

```

1 package DAO;
2
3 import java.util.List;
4
5
6
7
8 public interface CustomerDAO {
9     void save(Customer customer);
10    List<Customer> show();
11    void update(Customer customer);
12    void delete(int id);
13    void delete(String id);
14
15 }

```

Service

Jenis	nia
Satuan	3000
Status	nyuci
Harga	rp.10.000

ID	Jenis	Satuan	Status	Harga

Pada service, terdapat form **Jenis**, Quantity, Status, Harga. Untuk Jenis digunakan comboBox dengan nilai jenis-jenis pakaian, yaitu “pakaian, Seprei, Handuk, Jas, Gaun” .

Selanjutnya adalah **Quantity** untuk nilai satuan dan satuan nya, di mana jika Pakaian maka otomatis terisi Kg dan selain itu dengan satuan Helai.

Kemudian **Status** untuk mengetahui status laundry, menggunakan comboBox, dengan nilai “Dalam antrian, Sedang di cuci, Selesai”

Terakhir **Harga** yang otomatis terisi sesuai dengan harga x satuan. Dimana masing-masing harga yang di tetapkan adalah

```
private final double PAKAIAN_PRICE_PER_KG = 5000;  
private final double SEPREI_PRICE_PER_HELAI = 35000;  
private final double JAS_PRICE_PER_HELAI = 20000;  
private final double GAUN_PRICE_PER_HELAI = 50000;  
private final double HANDUK_PRICE_PER_HELAI = 10000;
```

dan untuk 'x quantity' menggunakan

```
case "Pakaian":  
    if (satuan.equals("Kg")) {  
        harga = quantity * PAKAIAN_PRICE_PER_KG;  
    }  
    break;  
case "Seprei":  
    if (satuan.equals("Helai")) {  
        harga = quantity * SEPREI_PRICE_PER_HELAI;  
    }  
    break;  
case "Handuk":  
    if (satuan.equals("Helai")) {  
        harga = quantity * HANDUK_PRICE_PER_HELAI;  
    }  
    break;  
case "Jas":  
    if (satuan.equals("Helai")) {  
        harga = quantity * JAS_PRICE_PER_HELAI;  
    }  
    break;  
case "Gaun":  
    if (satuan.equals("Helai")) {  
        harga = quantity * GAUN_PRICE_PER_HELAI;  
    }  
    break;  
default:  
    break;  
}
```

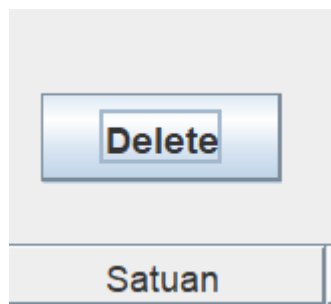
Tombol **Save** pada service

Berhasil menyimpan data yang telah di masukkan

Tombol **Update**

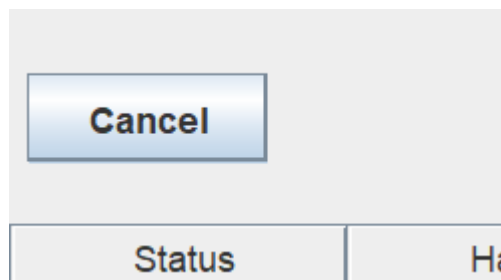
Berhasil merubah data yang telah di masukkan, semula Handuk menjadi Pakaian di nomor 10

tombol **Delete**



Berhasil menghapus data

Tombol cancel



Untuk men cancel isi form yang telah di isi sebelumnya

1.1 Penutup

Melalui praktikum ini, saya telah mempelajari dan mempraktikkan pembuatan fungsi CRUD menggunakan Java dan MySQL dengan menerapkan konsep Pemrograman Berorientasi Objek. Saya juga telah berhasil membuat GUI untuk mengelola data pengguna serta memanfaatkan fungsi DAO untuk modularitas dan pemisahan logika. Dengan pemahaman yang didapatkan, saya diharapkan mampu mengimplementasikan konsep-konsep ini pada proyek-proyek pengembangan perangkat lunak di masa mendatang, terutama yang melibatkan pengelolaan data berbasis database.